Module 2

Graphical Data Representation in Python

Data Analysis

Burkina Institute of Technology - L3 Computer Science
B. Tanguy KABORE

What we will see

- Why we draw charts and what makes a chart honest
- The Matplotlib figure: figure, axes, labels, legend
- Line, bar, histogram and scatter plots on concrete problems
- Plotting directly from a DataFrame and saving figures

Duration: 5 hours. Bring the notebook from Module 1, we reuse its data.

Why visualise data?

The point

A table of 1 000 numbers tells you nothing at a glance. A chart shows a trend, a gap between groups or an outlier in one second.

Comparison

A chart is like a map. The territory is the data. A good map leaves out details so you can see the road. A bad map hides the road.

Four questions, four chart types

Question	Chart
How does a value change over time?	Line plot
How do categories compare?	Bar chart
How are values distributed?	Histogram
Are two variables related?	Scatter plot

Choose the chart from the question, never from habit. Pie charts are rarely the right answer: humans compare lengths better than angles.

What is Matplotlib?

Simple definition

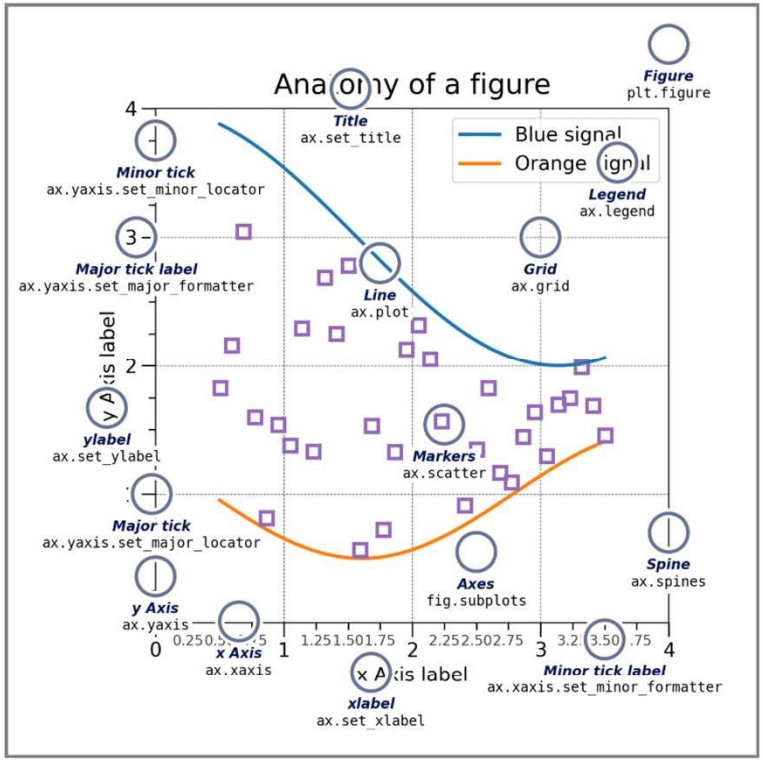
Matplotlib is the standard plotting library of Python. pandas, SciPy and most other libraries draw with it under the hood.

Convention

```
import matplotlib.pyplot as plt
```

`pyplot` is the module you will call. In Jupyter, the chart appears below the cell. In a script, you need `plt.show()` or `plt.savefig()`.

Anatomy of a figure



Source: Matplotlib documentation, "Quick start guide", https://matplotlib.org/stable/users/explain/quick_start.html - Matplotlib licence (BSD-compatible).

Figure and Axes

Figure

- The whole window or image
- Can contain several Axes
- Has a size in inches: `figsize=(8, 4)`
- You save it with `fig.savefig()`

Axes

- One chart with its x and y axis
- Where you call `.plot()`, `.bar()`, etc.
- Has a title, labels, legend
- Not the same thing as an "axis"

Most confusion in Matplotlib comes from mixing these two words.

Your first plot

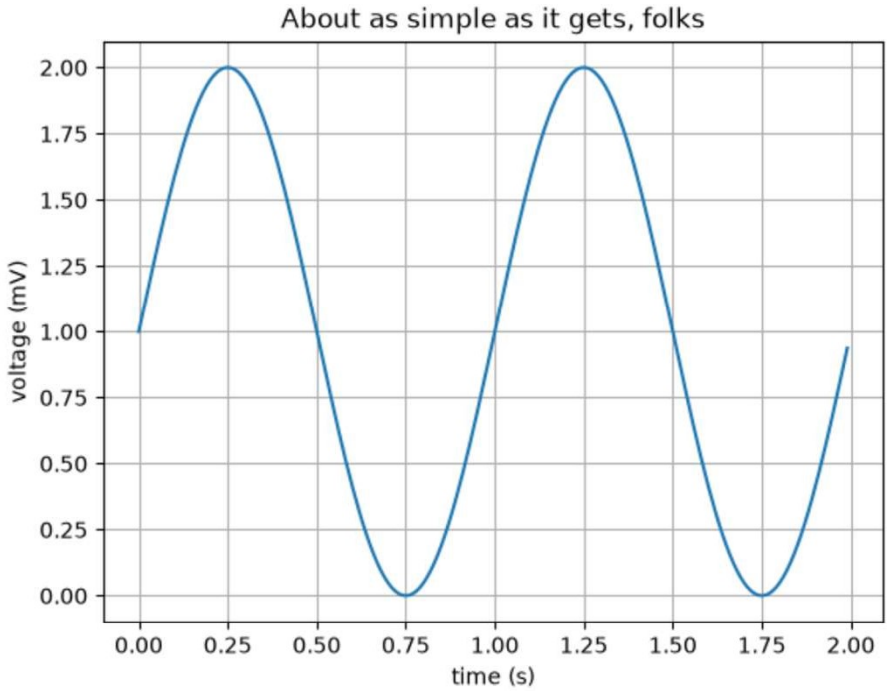
```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May"]
sales  = [20500, 24000, 11000, 18200, 22700]

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(months, sales, marker="o")
ax.set_title("Monthly sales 2026")
ax.set_xlabel("Month")
ax.set_ylabel("Amount (FCFA)")
plt.show()
```

`plt.subplots()` creates one figure and one axes. We always start like this.

Simple line plot



Source: Matplotlib gallery, "Simple Plot", https://matplotlib.org/stable/gallery/lines_bars_and_markers/simple_plot.html - Matplotlib licence.

The three lines you must never skip

```
ax.set_title("Monthly sales 2026")
ax.set_xlabel("Month")
ax.set_ylabel("Amount (FCFA)")
```

A chart without a title and units is a drawing, not a result. In your project, any figure missing one of these three loses points. Add `ax.legend()` as soon as there are two series.

Error: lengths do not match

```
ax.plot([1, 2, 3], [4, 5, 6, 7])
```

```
ValueError: x and y must have same first dimension,
but have shapes (3,) and (4,)
```

Cause: three x values, four y values. Often it comes from a `dropna()` applied to one column but not the other.

Fix: build x and y from the same DataFrame after cleaning, for example `ax.plot(df["date"], df["amount"])`.

Error: wrong keyword

```
ax.plot(x, y, colour="red")
```

```
AttributeError: Line2D.set() got an unexpected keyword argument 'colour'
```

Cause: Matplotlib uses American spelling: `color`, not `colour`. The same goes for `center`.

Fix: `ax.plot(x, y, color="red")`. Colour names must be valid too: `"red"`, `"tab:blue"`, `"#1f77b4"`. `color="rouge"` raises a `ValueError`.

Bar chart: comparing categories

```
depts = ["CS", "EE", "ME"]
avg   = [12.3, 14.5, 11.8]

fig, ax = plt.subplots()
ax.bar(depts, avg, color="tab:blue")
ax.set_title("Average Python grade per department")
ax.set_ylabel("Grade / 20")
ax.set_ylim(0, 20)
```

`set_ylim(0, 20)` matters here: a bar chart starting at 11 would make a 12.3 vs 14.5 gap look enormous. Bars must start at zero.

Horizontal bars and sorting

When category names are long, flip the chart:

```
s = df.groupby("dept")["python"].mean().sort_values()
fig, ax = plt.subplots()
ax.barh(s.index, s.values)
ax.set_xlabel("Average grade / 20")
```

Sorting the values before plotting turns a random-looking chart into a ranking that reads top to bottom.

Histogram: how values are spread

```
grades = df["python"].dropna()

fig, ax = plt.subplots()
ax.hist(grades, bins=10, range=(0, 20), edgecolor="black")
ax.set_xlabel("Grade / 20")
ax.set_ylabel("Number of students")
ax.set_title("Distribution of Python grades")
```

The `bins` parameter changes the story. Try 5, 10 and 20 on the same data before choosing. Too few bins hide the shape, too many show noise.

Reading a histogram

- A peak in the middle and symmetric tails: roughly normal, mean and median agree
- A long tail to the right: a few very high values pull the mean up
- Two peaks: probably two different populations mixed, for example two groups
- A spike at one value: look for a default value or a data entry rule

Always go back to the data when you see something odd. The chart raises the question; the table answers it.

Scatter plot: are two variables related?

```
fig, ax = plt.subplots()
ax.scatter(df["python"], df["stats"])
ax.set_xlabel("Python grade")
ax.set_ylabel("Statistics grade")
ax.set_title("Python vs Statistics")
```

If the points form an upward cloud, students good in one subject tend to be good in the other. We will measure this precisely in Module 4 with the correlation coefficient.

Several series on one chart

```
fig, ax = plt.subplots()
ax.plot(months, sales_2025, label="2025", linestyle="--")
ax.plot(months, sales_2026, label="2026", marker="o")
ax.legend()
ax.grid(True, alpha=0.3)
```

`label=` gives the name shown in the legend; without `ax.legend()` nothing is displayed. Keep to 4 or 5 series per chart, after that nobody can tell the lines apart.

Several charts in one figure

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].hist(df["python"].dropna(), bins=8)
axes[0].set_title("Python")

axes[1].hist(df["stats"].dropna(), bins=8)
axes[1].set_title("Statistics")

fig.tight_layout()
```

`axes` is now an array. `tight_layout()` stops titles and labels from overlapping. Use the same y-scale on both when you want to compare them: `sharey=True`.

Plotting directly from pandas

pandas wraps Matplotlib, so this works on any DataFrame or Series:

```
df.groupby("dept")["python"].mean().plot(kind="bar")
sales.set_index("date")["amount"].plot()
df.plot(kind="scatter", x="python", y="stats")
```

Each call returns the Axes, so you can still write `ax = ...plot(); ax.set_ylabel("...")`. Great for exploring quickly; switch to explicit Matplotlib for the final figures.

Error: nothing to plot

```
df["student"].plot()
```

```
TypeError: no numeric data to plot
```

Cause: the column contains text. pandas does not know how to put names on a y-axis.

Fix: plot a numeric column, or count the text values first: `df["dept"].value_counts().plot(kind="bar")`.

Saving a figure

```
fig.savefig("figures/sales_2026.png", dpi=150, bbox_inches="tight")
```

- `dpi=150` is enough for a report; use 300 for printing
- `bbox_inches="tight"` removes the white margins
- The extension chooses the format: `.png`, `.pdf`, `.svg`

If the `figures` folder does not exist, you get `FileNotFoundError: [Errno 2] No such file or directory: 'figures/sales_2026.png'`. Create it first with `os.makedirs("figures", exist_ok=True)`.

Making a chart honest

- Start bar charts at zero
- Use the same scale when comparing two charts side by side
- Say what the unit is: FCFA, %, students, degrees Celsius
- Do not truncate an axis to exaggerate a difference

A chart that misleads the reader is worse than no chart. In your project, I will look at the axes before I look at the colours.

Making a chart readable

- One message per chart; if you need two messages, draw two charts
- Label directly or use a legend, but not twelve colours
- Rotate long x labels: `ax.tick_params(axis="x", rotation=45)`
- Give the source of the data under the figure in your report

Exercise 1 - Chart choice (15 minutes)

Do it now

For each situation, say which chart you would use and why, then sketch it on paper:

1. Monthly electricity consumption of the campus over 3 years
2. Number of students per department this year
3. Ages of all students in the Institute
4. Hours of study per week versus final grade

Exercise 2 - First charts (30 minutes)

Do it now, with the grades table from Module 1

1. Bar chart of the average grade per department, y axis from 0 to 20
2. Histogram of the `average` column with 8 bins
3. Scatter plot of `python` against `stats`
4. Save the three figures in a `figures` folder as PNG

Each chart must have a title and labelled axes.

Practical work 2 - Visual report (1h30)

Goal

Build a one-page visual summary of a dataset: 4 figures, each with one sentence of interpretation.

Material

The student results from Practical work 1, or the monthly sales dataset provided in class (24 months, 3 product lines).

Work in pairs. Deliverable: notebook plus a PDF export of the figures.

Practical work 2 - Steps

1. Load and clean the data (reuse your Module 1 code)
2. Draw a line plot over time with one line per product or subject
3. Draw a bar chart of totals per category, sorted, starting at zero
4. Draw a histogram and a scatter plot that answer a question you state in a markdown cell

Practical work 2 - Finishing

1. Put the four charts in a 2 by 2 grid with `plt.subplots(2, 2, figsize=(12, 8))`
2. Add titles, axis labels, units and a legend where needed
3. Save the grid as `figures/report.png` at 150 dpi
4. Under each chart, write one sentence: what does the reader learn?

Practical work 2 - Checks before you submit

- No chart without a title and both axis labels
- Bar charts start at zero
- `fig.tight_layout()` called; nothing overlaps
- The notebook runs from top to bottom and recreates the PNG

Summary

What you know now

- Choose the chart from the question: line, bar, histogram, scatter
- `fig, ax = plt.subplots()` then `ax.plot()`, `ax.bar()`, `ax.hist()`, `ax.scatter()`
- Title, labels, units and legend are mandatory
- `savefig()` for reports; pandas `.plot()` for fast exploration

Next module

Module 3: Acquiring External Data in Python

References

- Matplotlib documentation, Quick start guide and gallery: https://matplotlib.org/stable/users/explain/quick_start.html and https://matplotlib.org/stable/gallery/index.html
- pandas, "Chart visualization": https://pandas.pydata.org/docs/user_guide/visualization.html
- A. Boschetti, L. Massaron, *Python Data Science Essentials*, 3rd ed., Packt, 2018, chapter 5 "Visualization, Insights, and Results"
- Examples tested with Matplotlib 3.10.1

Questions?

Find a chart in a newspaper or on social media this week that breaks one of our rules. Bring it to the next session.